Project 4:
Value Prediction

Version 1

ECE 721: Advanced Microarchitecture
Spring 2026, Prof. Rotenberg

Due: Friday, April 24, 2026, 11:59pm
This is a hard deadline. All project deliverables are due at this time.

- READ THIS ENTIRE DOCUMENT.
- Academic integrity:
    - **Outside of legal participation in message board Q&A (see the section on "Reasonable assistance", below), there shall be no cross-team collaboration whatsoever:** Each team must work individually on their project. There is zero tolerance for cross-team collaboration on the project in any form.
    - **Source code:** Each team must design and write their source code alone. They must do this (design and write their source code) without the assistance of any other person in ECE 721 or not in ECE 721. They must do this (design and write their source code) without searching the web for past semester's projects, computer architecture simulators or components of them, and the like, which is strictly forbidden. They must do this (design and write their source code) without looking at anyone else's source code, without obtaining electronic or printed copies of anyone else's source code, *etc*. Use of ChatGPT or other generative AI tools is prohibited. **Exception: For Section 2.2 of this project (competition phase), and ONLY Section 2.2 (NOT Section 2.1), you may leverage value predictor code from the First Championship Value Prediction.**
    - **Explicit debugging:** With respect to "explicit debugging" as part of the coding process (*e.g.*, using a debugger, inspecting code for bugs, inserting prints in the code, iteratively applying fixes, *etc*.), each team must explicitly debug their code without the assistance of any other person in ECE 721 or not in ECE 721.
    - **Sanctions:** The sanctions for violating the academic integrity policy are (1) a score of 0 on the project and (2) academic integrity probation for a first-time offense or suspension for a subsequent offense (the latter sanctions are administered by the Office of Student Conduct). Note that, when it comes to academic integrity violations, both the giver and receiver of aid are responsible and both are sanctioned. Please see the following RAIV form which has links to various policies and procedures and gives a sense of how academic integrity violations are confronted, documented, and sanctioned: RAIV form.
- Reasonable assistance: If a team has any doubts or questions, or if a team is stumped by a bug, the team is encouraged to seek assistance using both of the following channels.
    - Teams may receive assistance from the TA(s) and instructor.
    - Teams are encouraged to post their doubts, questions, and obstacles, on the Moodle message board for this project. The instructor and TA(s) will moderate the message

board to ensure that reasonable assistance is provided to the team. Other students are encouraged to contribute answers so long as no source code is posted.
* An example of reasonable assistance via the message board: Student A: "*I'm encountering the following assertion/problem, has anyone else encountered something like this?*" Student B: "*Yes, I encountered something similar and you might want to take a look at how you are doing XYZ in your renamer, because the problem has to do with such-and-such.*"
* Another example of a reasonable exchange: Student A: "*I'm unsure how to size my Free List based on the other parameters.*" Instructor/TA/Student B: "*You can reference the lecture notes on this topic but I'll also answer here. The key is that the PRF has a specified number of physical registers and, at any given time, a fixed number of these are committed registers. The number of committed registers is the number of logical registers. The committed registers cannot be free, ever. From that, you should be able to infer an upper bound required for the size of the Free List. For example, if the PRF size is 160 and the # logical registers is 32, then at most there can be 128 free registers. Again, also refer back to the lecture notes.*"

1. Project overview

1. Students will work in teams of two. Some students may already know their partners. For students who have not partnered with anyone yet, a google sheet will be shared with the class to facilitate finding partners. If there is an odd number of students in the class: for the one student who does not have a partner, the project scope will be adjusted in consultation with the instructor, unless the student wishes to keep the project scope the same. Exceptions to two-student teams will require approval of the instructor, a strong justification, and an appropriate plan for the project scope.
2. In this project, you will first implement a Stride Value Predictor (SVP) and support for value prediction in 721sim and validate it against the instructor's simulator. Then, you will improvise on the value predictor, its parameters, and value prediction support, to compete against other teams for highest harmonic mean IPC across selected benchmarks available in ECE 721.

2. Tasks and grading rubric

2.1. [80 points] Stride Value Predictor (SVP) with Value Prediction Queue (VPQ)

In the first phase of this project, you will implement the following:

1. Implement support for *making value predictions and confidence estimations in the Rename stage* and *injecting confident value predictions in the Dispatch stage* to break data dependencies between producers and consumers, following from the lecture notes on this aspect.

   An instruction is either eligible or ineligible for value prediction, based on (1) its type and (2) whether the simulator is configured to value predict its type. Also, the instruction must have a destination register (C_valid == true).[1] Use the following logic to determine eligibility:

[1] One might think that the instruction's type alone dictates whether the instruction has a destination register. There is a corner case, however, when the so-called destination register is x0 (logical register 0). Because the RISCV ISA specifies that x0 is hardwired to the immediate value #0, an integer ALU instruction or integer load instruction that has a logical destination register of x0 is like a NOP. 721sim's pipeline_t::decode() function sets C_valid == false in this case. Such an instruction is ineligible for value prediction.

First, add the following #define macros in pipeline.h (below the existing `IS_xxx()` macros):

```
#define IS_INTALU(flags)  ((flags) & (F_ICOMP))    // integer ALU instructions
#define IS_FPALU(flags)   ((flags) & (F_FCOMP))    // floating-point ALU instructions
```

Second, here is the eligibility code.

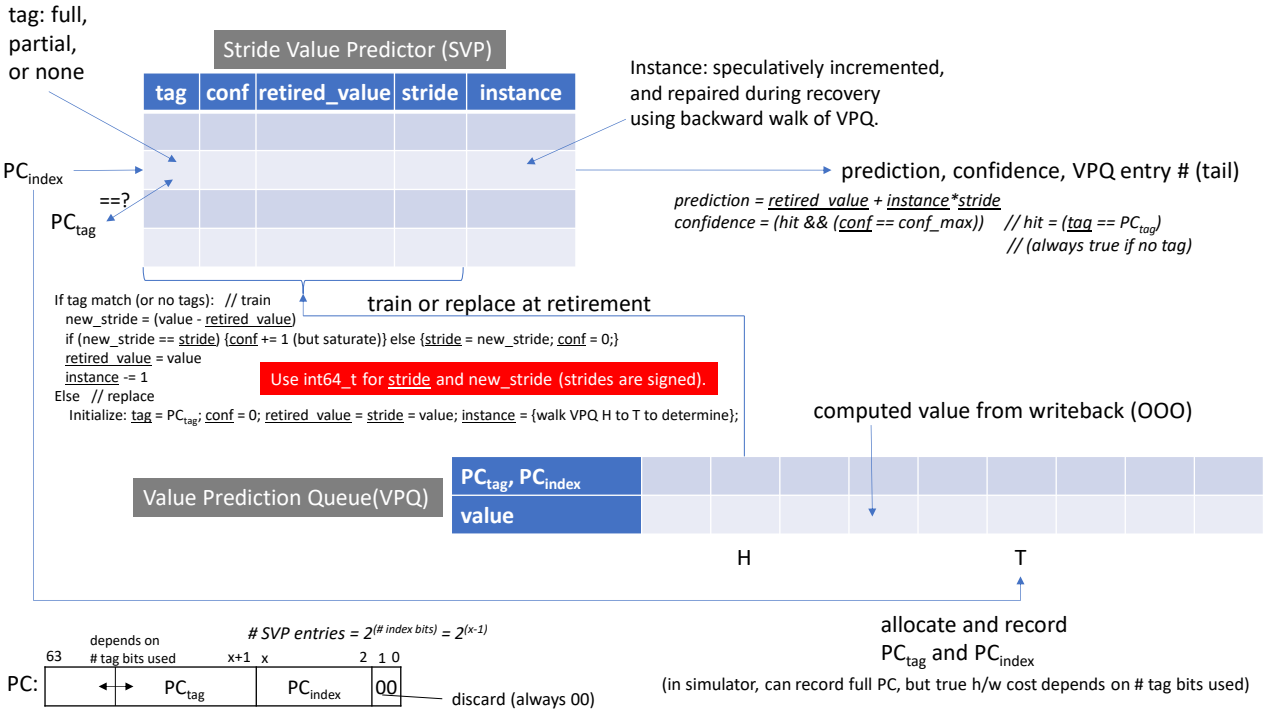
```
// This function checks value-prediction eligibility.
// predINTALU, predFPALU, and predLOAD are all "bool" types,
// and are configured to be true or false based on corresponding
// simulator arguments being 1 or 0, respectively.
bool pipeline_t::eligible(payload_t *pay) {
   if (!pay->C_valid)
      return(false);      // Any instruction without a destination register is ineligible.

   // If we reached this point, the instruction has a destination register.
   if (IS_INTALU(pay->flags))
      return(predINTALU); // instr. is INTALU type.  It is eligible if predINTALU is configured "true".
   else if (IS_FPALU(pay->flags))
      return(predFPALU);  // instr. is FPALU type.  It is eligible if predFPALU is configured "true".
   else if (IS_LOAD(pay->flags) && !IS_AMO(pay->flags))
      return(predLOAD);   // instr. is a normal LOAD (not rare load-with-reserv). It is eligible if predLOAD is configured "true".
   else
      return(false);      // instr. is none of the above major types, so it is never eligible
}
```

2. Implement a perfect value prediction mode, leveraging existing 721sim functionality that links each correct-path instruction in the pipeline simulator to its counterpart in the checker simulator (links are created in the Fetch stage, so they can be referenced in the Rename stage where value predictions are generated as per task 1 above). The counterpart's value is available in advance and can be used as a perfect value prediction. Perfect value prediction mode is useful for evaluating the speedup potential of value prediction, and for confirming the breaking of data dependencies implemented in step 1 above without needing support for value misprediction detection and recovery.

TIP: You can implement tasks 1 and 2 above to test breaking of data dependencies, without the need to implement value misprediction detection and recovery. After testing this aspect, you can move onto real value prediction, value misprediction detection, and recovery.

3. Implement a Stride Value Predictor (SVP) with Value Prediction Queue (VPQ), following from the lecture notes on this topic and depicted in the figure below.



4. Implement an oracle confidence mode. Like perfect value prediction mode, you will leverage the link between each correct-path instruction in the pipeline to its counterpart in the checker simulator. But instead of using the counterpart's value as a perfect value prediction, you will use it to check correctness of the real value prediction from your SVP, in advance. In oracle confidence mode, the SVP's confidence estimation is discarded and the real value prediction is only deemed confident if it matches the counterpart's value.

TIP: You can implement tasks 3 and 4 above to test breaking of data dependencies with a real value predictor (SVP), without the need to implement value misprediction detection and recovery (in oracle confidence mode, all incorrect value predictions are discarded and all correct value predictions are used). After testing that the SVP appears to be delivering some correct value predictions in oracle confidence mode, you can move on to the next step and run with real confidence. Ultimately, once everything is working, the utility of oracle confidence mode is in gauging the performance impact of real confidence.

5. Implement support for checking value predictions in the Execute/Load-Replay or Writeback/Load-Replay stage (see Appendix A, Comment 3) and posting value mispredictions in the Active List as they are detected (only if the instruction was actually value predicted), following from the lecture notes on this aspect. 721sim *already* has BR-1-like recovery support for value mispredictions: wait until posted value misprediction reaches the head of Active List, retire the value-mispredicted producer instruction, and full-squash all instructions in the pipeline.
6. Implement new command-line arguments to configure value prediction. Value prediction is enabled by specifying *one and only one* of N possible value predictors: perfect value predictor, SVP, and other value predictors you explore in the competition.

```
// Choose the perfect value predictor.
--vp-perf=1

// Choose and configure the Stride Value Predictor (SVP).
--vp-svp=<VPQsize>,      // number of VPQ entries
         <oracleconf>,   // 0: real confidence, 1: oracle confidence
         <# index bits>, // # SVP entries is 2**(# index bits)
         <# tag bits>,   // 0: no tag, 62-(# index bits): full tag, or
                         // somewhere between for partial tag
         <confmax>       // confidence threshold to be confident

// Choose and configure other predictors during competition phase.
--vp-???=?,?,…
--vp-???=?,?,…
```

If value prediction is enabled, by virtue of specifying *one and only one* of the value predictors, above, you must specify which instructions are eligible for value prediction using the following new command-line argument:

```
--vp-eligible=<predINTALU>, // 1: predict integer ALU instructions w/ dest.
                            // reg., 0: don't predict this instr. class
              <predFPALU>,  // 1: predict flt.pt. ALU instructions w/ dest.
                            // reg., 0: don't predict this instr. class
              <predLOAD>    // 1: predict load instructions w/ dest. reg.,
                            // 0: don't predict this instruction class
```

7. Your simulator must accurately tally and output the number of bytes for all value prediction table(s), for whichever value predictor is selected for a given run, based on its configuration. *You only need to do cost accounting for value prediction table(s).* For example, for the value predictor implemented in the first phase of this project, the simulator must output total bytes of storage for the SVP, whereas bytes of storage for the VPQ should be excluded.[2] The validation runs will show the format of this output for the latter value predictor. When you get to the competition phase, accurate cost accounting is essential because you must stay within a given storage budget for all value prediction table(s).
8. Your simulator must breakdown retired instructions into two top-level categories: "ineligible for value prediction" (vpmeas_ineligible) and "eligible for value prediction" (vpmeas_eligible). The top-level vpmeas_eligible category must be broken down further into five sub-categories. The five sub-categories classify the outcome of the prediction step:
    - "no prediction available" (vpmeas_miss) (for example, the instruction missed in the SVP table)
    - "confident and correct" (vpmeas_conf_corr) (possible IPC benefit)
    - "confident and incorrect" (vpmeas_conf_incorr) (value mispredictions)
    - "unconfident and correct" (vpmeas_unconf_corr) (lost opportunity)
    - "unconfident and incorrect" (vpmeas_unconf_incorr)

    Correct/incorrect is an indication of whether the value prediction was correct or incorrect, regardless of whether or not it was used (*i.e.*, regardless of confidence).
    Confident/unconfident is an indication of confidence, hence, whether or not the value prediction was used.
    The format of this output will be evident in the validation runs.

Validation runs are posted under the Project 4 topic in Moodle. You will self-grade your simulator using Gradescope, which will automatically compile and run the simulator, check for completion of grading plateaus as was done in prior projects, check IPC as was done in prior projects, and check the new outputs (storage cost accounting and retired instruction breakdown). All team partners should submit the team's simulator to Gradescope so that everyone receives a score.

Here is how Gradescope will score your simulator:

BASE:
If you pass at least one validation run with SVP and real confidence, ***BASE*** = 30. Otherwise, ***BASE*** = 0 - 30 based on manual inspection of the code (partial credit for key aspects implemented).

N: total number of validation runs.

validation_run_score$_i$:
Score for each validation run:

- 8: Simulator runs for > 1,000 instructions but < 10,000 instructions (faults, asserts, or deadlocks before 10,000 instructions).

[2] The rationale for excluding the cost of structures that track in-flight instructions (for retirement-time training and/or repairing value prediction tables on rollbacks), is that they are generally needed for any value predictor to work properly in the context of OOO pipelines.

- 15: Simulator runs for > 10,000 instructions but < 100,000 instructions (faults, asserts, or deadlocks before 100,000 instructions).
- 22: Simulator runs for > 100,000 instructions but < 1,000,000 instructions (faults, asserts, or deadlocks before 1,000,000 instructions).
- 29: Simulator runs for > 1,000,000 instructions but does not complete (faults, asserts, or deadlocks before completion).
- 36: Simulator runs to completion, but IPC differs from instructor's version by > 1%.
- 43: Simulator runs to completion and IPC matches within 1%, but one or more value prediction breakdown measurements differ by > 1% and/or storage cost accounting doesn't match exactly.
- 50: Simulator runs to completion, IPC matches within 1%, all value prediction breakdown measurements match within 1%, and storage cost accounting matches exactly.

The score *for this first phase* is calculated as follows (maximum of 80 points):

$$project\ score = BASE + \frac{\sum_{i=1}^{N} validation_run_score_i}{N}$$

Additional manual inspection of code:

The TAs will inspect all simulators to check for the following issues and make deductions for infractions:

- *Skirting VPU implementation aspects to get the simulator to run to completion.* Notice that 36 out of 50 points are for reaching the grading plateau "simulator runs to completion". The TAs will check that a VPU is implemented, that it is interfaced into the requisite pipeline stages, that value predictions are injected to break data dependencies, and that value mispredictions and recoveries are implemented and occurring in the validation runs.
- *Not using the VPU's VPQ to train the SVP.* It is a requirement of the project that the VPQ must be used to train the SVP, <u>not</u> PAY. The VPQ's functions are to (1) train the SVP in-order and non-speculatively at retirement, (2) facilitate initialization of the instance counter upon a replacement+allocation of an SVP entry, and (3) repair instance counters due to rollbacks (branch mispredictions, load violations, exceptions, value mispredictions). A matching project certainly demonstrates a correct VPQ implementation in terms of modeling its effect as a structural hazard (Rename stage stalls) and instance counter management. Nevertheless, we want to make sure students understand that PAY information is not literally available after the Writeback stage because the instruction is "drained from the execution lane" at that point, and its only residual existence is in FIFOs – Active List, SQ, LQ, BQ in fetch unit, VPQ in value prediction unit – as well as its value (if valid destination register) residing in the PRF. In the Retire stage, PAY should primarily be used for checking, assertions, and measurements – simulator infrastructure and not "silicon". (One exception: In addition to using PAY to manage VPU measurements in the Retire stage, you may use PAY to determine when to call your VPU training function in the Retire stage; it is understood that this is equivalent to adding a flag in the Active List to signal VPU training.)
- *Use of good_instruction to gate actions in "real value prediction + real confidence" mode*. The "good_instruction" flag should only be used to facilitate "perfect value prediction" mode and "real value prediction + oracle confidence mode". It must NOT be used in any code

supporting “real value prediction + real confidence” mode. For example, even instructions on the incorrect path, that will ultimately be squashed by a prior mispredicted branch, must be eligible for value prediction and breaking data dependencies.

- Related to the previous bullet: *Use of the value from the debug buffer (actual->a_rdst[0].value) to check for value mispredictions.* This is just wrong. When a value-predicted producer instruction executes, it should compare its computed value to its predicted value, period (no matter what the mode of the simulator is).

2.2. [20 points] Competition

You must complete the first phase of the project (Section 2.1) before you are eligible for the competition phase. By "complete the first phase", we mean "**pass all validation runs in Gradescope (all "Passed Tests" and no "Failed Tests", *i.e.*, autograder score = 50 points) without infractions (infractions: see 'Additional manual inspection of code')**". If your team enters the competition, you will use your value prediction simulator to see how far you can push performance within a fixed storage budget for value prediction.

2.2.1. Example strategies

You are free to explore the following strategies and possibly others. **These are just suggestions and ultimately you decide how to budget your time in the competition.**

- This is a strong suggestion: Since the VPQ and/or other structures for in-flight instruction tracking do not count against the fixed storage budget, make sure you size these adequately such that they are not a frequent structural hazard (or never a structural hazard). Add a counter that counts the number of cycles during which the Rename stage was stalled, solely because the VPQ/other in-flight structure didn't have sufficient space for all value-prediction eligible instructions in the rename bundle. You can use this performance counter to diagnose stalls caused by an undersized VPQ/other in-flight structure. Of course, alternatively, you can sweep the size(s) of these structure(s) and look at IPC, or analytically bound the size(s) based on the pipeline configuration, *etc*.
- Perhaps explore the design space of your SVP within the fixed storage budget. For example, you can tradeoff the number of bits per SVP entry (*e.g.*, full, partial, or no tags) with more or fewer SVP entries; you can experiment with "conf_max"; you can experiment with which instruction classes to predict (some instructions may be more critical to performance than others, and the fewer eligible instructions, the lesser the competition for SVP entries); *etc*.
- Perhaps improvise on your SVP, for example: add some global branch history context to specialize strides and attempt to address the problem of skipped instances caused by control flow; I believe there are prior works that maintain multiple strides per PC and select among them; *etc*. It may be wise to characterize value patterns before carrying out time-consuming improvisations. It may be wise to first try CVP-1 predictors or other predictors from the literature since their authors carried out studies (see below).
- **Perhaps try out one or more value predictors from the First Championship Value Prediction (CVP-1).** Contestants of CVP-1 provided their source code. Keep in mind that these predictors were integrated into a specific trace-driven simulator and will need to be ported to 721sim. In particular, CVP-1's value predictor API is peculiar in that contestants enjoyed the illusion of speculatively updating value predictor context as needed, without the burden of repairing context after rollbacks (owing to the trace-driven nature of the CVP-1 simulator). In other words, I doubt they model machinery akin to the VPQ. Thus, some 721sim integration effort is required. Moreover, there is no guarantee the CVP-1 predictors will perform similarly in the context of 721sim and our SimPoints. Once deployed, experiment with relevant parameters and policies within the fixed storage budget.
- Perhaps implement CVP-1 value predictors from scratch rather than use their source code as a starting point, for more control and a cleaner 721sim-compatible implementation. Once deployed, experiment with relevant parameters and policies within the fixed storage budget.

- Perhaps research references in the CVP-1 bibliography and other value predictor papers, and explore one or more of these value predictors. If and when you deploy these value predictors, experiment with relevant parameters and policies within the fixed storage budget.
- Perhaps explore your own ideas if you uncover insights from ECE 721 SimPoints.
- **Perhaps implement immediate value misprediction recovery, using VR-5 instead of VR-1.** There is interplay between recovery performance and aggressiveness of value prediction. If the misprediction penalty is reduced, it might be possible to decrease the confidence threshold → more aggressive value prediction → increase value prediction coverage ("confident and correct" up) at the price of more mispredictions ("confident and incorrect" up). For fair play in the competition, existing BR-5 resources must remain fixed. That is, GBM bits / branch IDs / checkpoints must be shared among branches and value-predicted instructions. Improvising further, you can explore using dual confidence levels: high and medium confidence. High-confident value-predicted instructions don't consume VR-5 resources and use VR-1 recovery if they are mispredicted, whereas medium-confident value-predicted instructions consume VR-5 resources. You can also play with different policies when VR-5 resources are unavailable for value-predicted instructions in the rename bundle: stall rename vs. class it for VR-1 recovery vs. don't value predict it.

2.2.2. Key competition rules

- There is one baseline superscalar processor configuration. This configuration will be posted under the Project 4 topic of Moodle. The choice of baseline superscalar processor is important because of the interplay between window size, superscalar width, and value prediction.
- The value prediction mode is only "real value prediction + real confidence".
- There is a single fixed storage budget for the value predictor. The fixed storage budget will be posted under the Project 4 topic of Moodle. You must painstakingly account for and document all bytes of storage in the value prediction table(s). In-flight instruction tracking structures (*e.g.*, the VPQ) do not count against the fixed storage budget. Your simulator must print out detailed storage cost accounting (as you did for SVP in Phase 1).
- A given team will submit a "final entry" for evaluation by the TAs. The final entry is comprised of (1) the team's simulator, (2) flags that select the final value predictor and its final configuration, (3) a spreadsheet of final benchmark IPCs and harmonic mean IPC (calculation explained below), for corroboration with the TAs' runs with the final entry, and (4) final storage cost (total, and broken down among key components), for corroboration with the TAs' examination of storage cost and to check that the final entry is within budget. It is my expectation that you have one configurable simulator that has the SVP+VPQ from Section 2.1 and at least the final value predictor from Section 2.2 (if not all value predictors explored in Section 2.2).
- If your simulator fails to complete one or more benchmarks (faults, asserts, or deadlocks before completion), then you may not use that simulator configuration as your final entry in the competition. Either fix the bug or use a different configuration that runs to completion on all benchmarks.
- You are prohibited from *a priori* configuring your simulator based on the benchmark or benchmark SimPoint that is being run. That is, the modeled superscalar processor (and its value predictor, value prediction support, *etc*.) does not have *a priori* knowledge of the

benchmark being run nor *a priori* knowledge of that benchmark's preferred value predictor parameters. If you want to dynamically adapt the value predictor's parameters to program behavior, you need to invent hardware to achieve such adaptivity and include its storage cost in the fixed storage budget. Moreover, such adaptivity must be limited to policies, thresholds, *etc*.; on the other hand, it is unfair to assume storage can be shifted among different subcomponents (assumption of reconfigurable storage is prohibited in this competition). Hybrid value predictors, multiple-table value predictors, *etc*., are acceptable because (1) they are intrinsically adaptive, (2) all subcomponents are accounted for in the fixed storage budget, and (3) storage is not shifted among subcomponents (each subcomponent is fixed).

- Recovery is limited to squash-based recovery, either VR-1 (721sim already implements VR-1 for value misprediction recovery), VR-5, or hybrids. Complete or selective re-execution is prohibited for two reasons: (1) it takes a long time to implement re-execution based recovery, plus it is challenging to get right (particularly selective re-execution), so I advise against it, (2) some implementations require more storage throughout the pipeline (dependence vectors, replay buffer, *etc*.), and (3) aside from the VR-1 vs. VR-5 aspect, the focus of the competition is the value predictor.

2.2.3. Competition scoring

The competition points are divided into two parts:

- IPC rank [0-10 points]: A given team will be awarded 0-10 points based on their final entry's performance. The performance metric is the harmonic mean of IPCs of all benchmarks. (Formula below. Excel has a built-in HARMEAN() formula.) The mapping of percentile rank (percentage of contestants below you) and/or harmonic mean IPC, to points, is yet to be determined, and may be linear or non-linear, continuous or discrete, and so forth.

$$\text{harmonic mean IPC} = \frac{N}{\sum_{i=1}^{N} \left(\frac{1}{IPC_i}\right)}$$

- Recorded presentation [0-10 points]: Every competing team must create a powerpoint presentation that describes additional value predictors that were attempted, their total and per-subcomponent storage costs, their design space aspects that were explored, *etc.*, and the results and insights of these efforts. The team must record a 10-minute presentation (*e.g.*, you can use Zoom to record). If you can't fit everything you explored, prioritize your final entry and other competitive design points. The instructor and TAs will evaluate the recorded presentation and assign 0-10 points based on the quality of the team's research. Recordings that exceed the 10-minute time limit will not be considered (0 points).

2.2.4. HPC

The Project 4 topic in Moodle will have information about using NCSU's High-Performance Computing (HPC) cluster for the competition phase. HPC can support the large number of jobs that teams will need to submit during the competition. Students will be provided HPC accounts and the TAs will install the benchmarks and two scripts for (1) submitting all benchmarks with one command (for a 721sim configuration of your choice) and (2) collecting results from the latter experiment.

Appendix A:
Comments about implementing value prediction in 721sim

Comment 1

The VPQ should be modeled as a structural hazard. That is, you should model stalling if there aren't enough VPQ entries.

Details of where to stall:

Value Prediction Module #4 (Slide 42): The Rename stage is the value prediction stage (generate value predictions and confidence estimations).
Value Prediction Modules #3a (Slide 16) and #3b: In the Rename stage, for each VP-eligible instruction, we (1) attempt value prediction and (2) allocate a VPQ entry at the tail, used for training later at retirement.

Therefore, the Rename stage stalls if the VPQ doesn't have enough free entries for ALL VP-eligible instructions in the rename bundle. So add another stall condition to the Rename stage: count how many VP-eligible instructions are in the rename bundle, and check for at least that many free VPQ entries. Only when it is clear that the Rename stage won't stall (adequate branch checkpoints, adequate physical registers, adequate VPQ entries for all VP-eligible instructions, and Dispatch isn't stalled) and thus the rename bundle will advance, will you proceed with renaming and value-predicting instructions in the bundle.

Comment 2

Our implementation of value prediction breaks data dependencies by making it appear that the value-predicted instruction executes early, in the Dispatch stage:

Value Prediction Module #4 (Slide 43): For a value-predicted instruction, we set the ready bit of its destination physical register (instead of clearing the ready bit) and write the predicted value into its destination physical register in the PRF.

If you don't change anything else in 721sim, it is possible for the following assertions to fire:
issue_queue.cc:112: assert(!q[i].A_ready);
issue_queue.cc:121: assert(!q[i].B_ready);
issue_queue.cc:130: assert(!q[i].D_ready);

Explanation: Consumers of the value-predicted instruction will be dispatched into the IQ with their dependent source(s) initialized as (speculatively) "ready". After the value-predicted instruction issues, it will call issue_queue::wakeup() to wakeup dependent source operands in the IQ. If a dependent instruction is still in the IQ (*e.g.*, it wasn't selected for issuing despite being ready, or maybe it is waiting on a non-predicted source operand), one of the above assertions will fire because the predicted source operand was marked initially as ready.

Possible fixes:

The best fix (and the one I recommend) is to NOT call issue_queue::wakeup() for a value-predicted instruction, since this step is redundant for it: it precluded the need for this step by virtue of initializing its PRF ready bit to true. Note that issue_queue::wakeup() is called from multiple places in the backend, due to different handling of 1-cycle producers vs. multi-cycle producers, and specialized load handling (for both non-stalled loads and stalled-and-replayed loads).

A simpler fix is to comment-out the above assertions, or disable them when 721sim enables value prediction. I don't recommend this fix because these assertions are valuable for detecting bugs affecting non-predicted source operands, as we observed in Projects 2 and 3.

Comment 3

Background: Recall from Project 3, that 721sim models the effect of PRF bypasses – without the benefit of "wires" in C++ – by writing the PRF in the final cycle of execution rather than in the Writeback stage (lesson: this is just a *simulator hack* to model the effect of PRF write ports and bypasses in Writeback).

There are two stages where you can check value predictions:

- *RECOMMENDED: In the final stage of Execute/Load-Replay for a given value-predicted instruction.* The advantage of checking the value prediction here, is that you can control both writing the PRF and setting the value misprediction flag in the Active List based on the result of the check. While it is correct to always write the PRF with the computed (correct) value, regardless of the prediction being correct or incorrect, NOT writing the PRF for a correctly-value-predicted instruction models the PRF write port bandwidth sharing between Dispatch and Writeback as discussed in Slide 45 of Value Prediction Module #4. In other words, you can test your learning of Slide 45 by only writing the PRF with the computed value if either the instruction wasn't value-predicted (Slide 45: !x) or if it was value-predicted and the prediction was incorrect (Slide 45: !y). Again, your simulator will be correct if you unconditionally write the PRF, but you will miss an important learning opportunity: avoiding redundant PRF writes for correctly-value-predicted instructions. Note that renamer::write() is called from multiple sites in execute.cc. You must implement the value prediction check at all these call sites, including several places in pipeline_t::execute() and replayed loads in pipeline_t::load_replay() (loads that stalled in the LSU and subsequently get unstalled).
- *In the Writeback/Load-Replay stage.* If you choose to implement value prediction checking here, there are two issues you need to be aware of: (1) You will miss-out on implementation of Value Prediction Module #4, Slide 45. This is due to the 721sim hack of modeling the PRF bypasses by calling renamer::write() in the final cycle of Execute instead of Writeback. (2) Unfortunately, 721sim doesn't currently model load replays exactly the way I would like it to, and there are implications for you (if you choose value prediction checking in Writeback):

- Real hardware: A stalled load gets replayed from the LQ by arbitrating with AGEN for access to an LSU port. If AGEN is idle, the load is reinjected into the LSU pipeline. If it succeeds, it will flow through the Writeback stage.
- 721sim: 721sim assumes a dedicated pathway for re-accessing the LSU. If the replayed load succeeds, it circumvents 721sim's pipeline_t::writeback() function (unfortunately). Essentially, the dedicated pathway includes a dedicated writeback point in the 721sim source code. What this means for you: you must not only check the value prediction in pipeline_t::writeback(), but also pipeline_t::load_replay(), because loads that complete in pipeline_t::load_replay() do not pass through pipeline_t::writeback() (unfortunately).